Markel Corbatón Pérez y Daniel Fernández Pérez

NAMED PIPES

Qué son:

Un pipe (tubería) es un mecanismo de comunicación entre procesos. Un Named Pipe tiene un nombre, así que dos programas independientes pueden encontrarse sin conocerse previamente, con solo usar el mismo nombre.

Características principales:

-Comunica procesos distintos, normalmente en el mismo equipo.

-Puede ser bidireccional (PipeDirection.InOut): por el mismo canal van la petición y la respuesta.

-Transporta bytes o texto, no objetos. Por eso se serializa a JSON.

-Funciona con un modelo servidor/cliente: uno crea el tubo y espera, el otro se conecta.

-Cada instancia del servidor atiende a un cliente a la vez.

Clases de C# (System.IO.Pipes):

-NamedPipeServerStream - crea la tubería y espera conexiones(App2)

-NamedPipeClientStream - se conecta a un tubo existente(App1)

-StreamReader/StreamWriter - leen y escriben texto sobre el tubo

-PipeDirection.InOut - tubería de entrada y de salida

-JsonSerializer - convierte objetos a Json y viceversa

App 1: (Gestión de restaurante, WPF + BD):

El usuario selecciona un pedido y pulsa “Generar factura”. Actúa como **cliente**: serializa las líneas del pedido a JSON y las envía.

App 2: (Facturación, WPF sin BD):

Actúa como **servidor**. Recibe el JSON, lo deserializa, valida, calcula subtotal, descuento, IVA y total, anota la petición en su registro (DataGrid + JSON) y devuelve la respuesta.

(App 1 ⇄ Named Pipe + JSON ⇄ App 2 → BD).

Paso	APP 1 (cliente)	Tubo	APP 2 (servidor)
1			Crea el tubo "FacturacionPipe"
2			WaitForConnectionAsync → espera...
3			SIGUE ESPERANDO
4	Construye SolicitudFactura (objeto C#)		SIGUE ESPERANDO
5	Serialize → texto JSON		SIGUE ESPERANDO
6	ConnectAsync	→ Conexión →	Se conecta alguien, despierta
7	WriteLineAsync(json)	→ JSON de la solicitud →	ReadLineAsync → recibe el texto
8	ReadLineAsync → espera...		Deserialize → objeto SolicitudFactura
9	Esperando		Procesar: valida y calcula
10	Esperando		Serialize → texto JSON de respuesta
11	Recibe el texto	← JSON de la respuesta ←	WriteLineAsync(jsonRespuesta)
12	Deserialize → objeto RespuestaFactura		
13	Guarda en la tabla FACTURAS		
14	Cierra el tubo		Cierra el tubo y vuelve al paso 1

Por qué hace falta comunicar procesos. Cada programa en ejecución es un proceso con su propia memoria, y ninguno puede leer las variables del otro. Si dos apps necesitan intercambiar datos, el sistema operativo tiene que ofrecerles un canal. A eso se le llama comunicación entre procesos (IPC).

Ciclo de vida de la conexión. El servidor crea el tubo y espera. El cliente se conecta y se abre el canal. Uno escribe un mensaje y el otro lo lee, y después responden en sentido contrario. Al terminar se cierra el tubo, y el servidor crea otro para atender al siguiente cliente.

Por qué asíncrono. Esperar una conexión o un mensaje es una operación que bloquea hasta que ocurre algo. Si eso pasa en el hilo de la ventana WPF, la interfaz se congela. Con async/await el programa espera sin bloquear ese hilo.

Direcciones del tubo. In solo recibe, Out solo envía e InOut hace las dos cosas. Como vuestra comunicación es de petición y respuesta, el mismo tubo tiene que enviar y recibir. Por eso en las dos apps se usa InOut.

Cómo se identifica el tubo. Se identifica por su nombre, que debe ser exactamente igual en cliente y servidor. En el cliente, el "." indica que el servidor está en este mismo equipo. En Windows, el tubo vive en una ruta especial del sistema, \\.\pipe\NombreDelTubo.

Serialización. Serializar es convertir un objeto en texto y deserializar es el camino inverso. Hace falta porque el tubo solo transporta texto o bytes. JSON es un formato legible, estándar y que entiende cualquier lenguaje.

Pipe anónimo vs. Named Pipe. El anónimo no tiene nombre y solo sirve entre un proceso y los que él mismo lanza, normalmente en un solo sentido. El Named Pipe tiene nombre, así que procesos que no se conocen pueden encontrarlo. Como vuestras apps son independientes y se abren por separado, necesitáis uno con nombre.

Cómo se delimitan los mensajes. Un tubo es un flujo continuo y no marca dónde acaba un mensaje. Vosotros lo resolvéis con una línea por mensaje: WriteLineAsync añade un salto de línea y ReadLineAsync lee hasta encontrarlo. Si el JSON llevara saltos de línea, se leería cortado.